



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение высшего
образования
«МИРЭА Российский технологический университет»
РТУ МИРЭА

**Оптимизация культурооборота на рассадно-овощном
тепличном комбинате**

Выполнил студент 2 курса группы ЭФБО-04-24
Валекжанин Владимир Сергеевич

г. Москва
2026 год

Содержание

Содержание.....	2
Введение.....	3
1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ.....	5
1.1 Анализ предметной области.....	5
1.2 Анализ аналогов разрабатываемого решения.....	5
1.3 Описание требований к разрабатываемому решению.....	5
1.3.1 Функциональные требования.....	5
1.3.2 Нефункциональные требования.....	5
2 ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ.....	6
2.1 Выбор инструментов или средств реализации проекта.....	6
2.1.1 Сводная таблица выбранных инструментов.....	6
2.1.1 Выбор СУБД.....	7
2.1.2 Выбор пакетного менеджера.....	8
2.1.3 Назначение библиотек тестирования.....	8
2.1.4 Выбор ORM-системы.....	9
2.1.5 Выбор хостинга кода.....	11
2.1.6 Выбор HTTP-обработчика.....	11
2.2 Разработка проекта решения.....	11
2.2.1 Упрощенная система ролей.....	11
2.2.2 Диаграммы.....	12
2.2.2.1 Диаграмма сценариев использования.....	12
2.2.2.2 Entity-Relationship диаграмма (ERD).....	13
2.2.2.3 Диаграмма последовательности.....	14
3 ПРАКТИЧЕСКАЯ ЧАСТЬ.....	15
3.1 Разработка составных элементов.....	15
3.2 Установка и настройка.....	15
3.3 Тестирование.....	15
3.4 Ввод в эксплуатацию.....	15
3.5 Разработка сопроводительной документации.....	15
Заключение.....	16
Список использованных источников.....	17
Приложения.....	18

Введение

У агрономов защищенного грунта широкий круг обязанностей: от разработки технологических карт и контроля фитосанитарного состояния до учета ресурсов и финансовых затрат. Промышленное овощеводство — это высокотехнологичный процесс, где ошибка в планировании сроков на 5 дней может привести к потере до 20% урожайности. Мы решили создать систему, которая автоматизирует расчет потребностей и оптимизирует графики работ, отвечая на главный вопрос бизнеса: «Как максимально эффективно использовать каждый метр теплицы, чтобы получить прибыль?»

Данная работа актуальна, так как в сфере российской аграрной промышленности нет аналогичных продуктов. Наши конкуренты либо не имеют такой функциональности, либо она реализована частично.

Современный рынок разработки программного обеспечения обязывает разработчиков программного обеспечения не только быть специалистами своей области, но и ориентироваться в большинстве смежных технологических областей. Данная работа посвящена разработке современного программного обеспечения в соответствии с поставленной задачей и темой работы. Такая деятельность, помимо формирования практического опыта, позволяет сформировать новое видение классических задач по разработке программных комплексов и программного обеспечения.

Наша аудитория — агрономы и директора рассадно-овощных тепличных комбинатов, которые хотят ускорить конвейер культурооборота без ущерба качеству.

Цель работы заключается в разработке ПО для автоматизированного администрирования и оптимизации производственных циклов в условиях промышленного культурооборота.

Объектом данного исследования является промышленный культурооборот в условиях тепличного комбината.

Предметом исследования является цифровое моделирование производственных циклов защищенного грунта для обоснованного распределения площадей в условиях ограниченных ресурсов предприятия.

В качестве основных **методов исследования** применены анализ, синтез, сравнение и моделирование. Практическая реализация поставленной задачи соответствует основным подходам к разработке программного обеспечения.

Информационная база исследования содержит онлайн-статьи независимых авторов, справочники федерального назначения по ведению аграрного хозяйства.

Задачи работы:

- Провести анализ предметной области;
- проанализировать аналоги разрабатываемого решения;
- описать требования к разрабатываемому решению;
- выбрать инструменты и средства для реализации проекта;
- разработать проект решения;
- разработать программный продукт;
- составить отчет по работе;
- представить отчет к защите и ответить на вопросы по проекту.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Анализ предметной области

1.2 Анализ аналогов разрабатываемого решения

Ниже приведена сводная таблица 1.1, в которой мы рассмотрели нескольких конкурентов нашего решения.

Таблица 1.1 — сравнительный анализ конкурентов по наличию тех или иных функций

Конкурент	Управление ТК	Советы по оптимизации производства	Поддержка IoT-систем	Визуальное расположение объектов на карте
https://plantix.net	Нет	Нет	Нет	Нет
https://onesoil.ai	Нет	Частично	Нет	Да
https://agrobases.ru	Нет	Нет	Нет	Нет
https://agro-scout.ru	Нет	Нет	Частично	Да
agrar.io	Да	Да	Да	Да

1.3 Описание требований к разрабатываемому решению

1.3.1 Функциональные требования

1.3.2 Нефункциональные требования

- Наличие доступной пользовательской документации непосредственно на самой платформе;

2 ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ

2.1 Выбор инструментов или средств реализации проекта

2.1.1 Сводная таблица выбранных инструментов

В таблице 2.1 приложены все технологии, которые мы используем в нашем проекте, и их назначение.

Таблица 2.1 - сводная таблица используемых технологий

Технология	Назначение
oxlint, oxfmt	Линтинг и форматирование соответственно
Vitest	Фреймворк юнит-тестирования
react-testing-library	Вспомогательная библиотека для юнит-тестирования React-компонентов
Vite + @vitejs/plugin-react	Сборщик кода (bundler)
Storybook	Скриншотные тесты
Playwright	E2E тестирование
pnpm	Пакетный менеджер с упором на кеширование и поддержку монорепозитория
nx	Платформа управления монорепозиторием
react	Библиотека для поддержки JSX-компонентов
react-router	Маршрутизатор для SPA
react-admin	Библиотека готовых компонентов для административных систем
dhtmlx-gantt	Диаграмма Ганта
leaflet	Интерактивная карта местности
terra-draw + leaflet-adapter	Готовое решение для рисования и редактирования полигонов на карте местности. Интегрирован с leaflet через leaflet-adapter
TypeScript	Статический типизатор JS

PostgreSQL	Система управления базами данных (СУБД)
Docker + docker compose	Контейнеризация
Gitlab	Хостинг кода
Gitlab CI/CD	Пайплайн CI/CD
ExpressJS	Асинхронный маршрутизатор запросов
Zod	Валидация данных и поддержка строгих контрактов
Prisma ORM	Проектирование схемы БД, миграции, генерация клиентских типов
prisma-generator-plantuml-erd, prisma-generator-express	Генерация CRUD на ExpressJS и генерация ER-диаграммы
XiorJS	Абстракция над Fetch API. Отправка запросов и обработка ответов по HTTP/2.0
husky	git-хуки (CI/CD)
git	Система контроля версий (VCS)

2.1.1 Выбор СУБД

Мы выбрали СУБД PostgreSQL 18-ой версии. На то несколько причин: во-первых, наш проект требует поддержки геополитических данных формата geoJSON. PostgreSQL имеет плагин PostGIS, который предоставляет эту поддержку. Также этот плагин добавляет вспомогательные методы для поиска площадей полигонов — например, теплиц или самого предприятия, что исключает потребность самостоятельно реализовывать эту логику на бекенде.

Использование MongoDB нецелесообразно в контексте системы автоматизированного планирования культурооборота. Реляционная архитектура обеспечивает необходимую строгость связей, транзакционную надежность, эффективную обработку сложных запросов и прямую совместимость с математическими моделями оптимизации. Выбор реляционной СУБД соответствует отраслевым стандартам разработки модулей управления производством и информационных систем.

2.1.2 Выбор пакетного менеджера

Мы выбрали `pnpm`, потому что он заточен кеширование зависимостей и на работу с монорепопозиториями. `pnpm` не имеет таких возможностей.

Мы не рассматривали `yarn`, `bun` и `deno`, так как они не имеют такой гибкой поддержки монорепопозитория, как `pnpm`. Также изучение и интеграция данных инструментов требует времени.

2.1.3 Назначение библиотек тестирования

Как видно в таблице 2.1, мы выбрали сразу несколько инструментов для тестирования. Мы решили так сделать, так как это стандарт индустрии — данные инструменты покрывают свои ниши тестирования, и использования только части из них не достаточно для полного покрытия.

`Storybook` необходим для проверки внешнего вида интерфейса на разных платформах.

`Vitest` позволяет писать `unit`-тесты на фронтенде и бекенде, к тому же он очень быстрый и интегрирован с `vite`. Но одного `vitest` не достаточно - для тестирования `React`-компонентов целесообразно использовать соответствующую вспомогательную библиотеку, с чем справляется `react-testing-library`. Запускается он в среде `jsdom` (а не в `browser`), так как работа с браузером покрыта `E2E`-тестами, а лишняя интеграция с эмуляцией окружающей среды браузера привносит свои краевые случаи и полностью лишает нас скорости `vitest`.

`Playwright` также интегрирован в экосистему `vite`, и является безоговорочным лидером в `E2E`-тестировании.

2.1.4 Выбор ORM-системы

В таблице 2.2 мы рассматривали несколько ORM-систем, включая Prisma, TypeORM, Drizzle, MikroORM и Sequelize.

Таблица 2.2 - сравнительная таблица популярных ORM-решений

ORM	Поддержка плагинов	Типизация	Транзакционная согласованность	Валидация	Производительность	Управление миграциями	Необходимость в драйвере
Prisma ORM	Да	Полная	Хорошая	Полная	Низкая	Декларативное	Нет
TypeORM	Да	Полная	Полная	Хорошая	Низкая	Императивное	Да
Drizzle ORM	Ограниченная	Полная	Полная	Полная	Высокая	Декларативное	Да
MikroORM	Да	Полная	Полная	Хорошая	Средняя	Императивное	Да
Sequelize	Да	Очень слабая	Полная	Хорошая	Низкая	Императивное	Да

Как видно, Prisma имеет несколько преимуществ перед другими ORM. Рассмотрим некоторые особенности Prisma подробнее.

Во-первых, схема БД прописывается в одном или нескольких файлах с расширением `.prisma`. Prisma имеет декларативный синтаксис, который напоминает SQL. Хотя этот синтаксис и ограничен и не поддерживает триггеры и проверку входных данных (помимо NOT NULL, UNIQUE), но это можно нивелировать на стороне бекенда.

Также Prisma изначально поддерживает миграции (система Prisma migrations) и генерацию клиентских типов (`@prisma/client`).

Prisma поддерживает пользовательские генераторы. Мы используем `prisma-generator-express` — генератор CRUD на ExpressJS для каждой сущности в схеме, и `prisma-generator-plantuml-erd` — генератор ER-диаграммы в форматах PlantUML, Markdown и txt.

2.1.5 Выбор хостинга кода

Мы выбрали Gitlab, а не Github, потому что наш проект имеет Gitlab CI/CD пайплайн. Этот пайплайн поддерживает только Gitlab.

2.1.6 Выбор HTTP-обработчика

Мы выбрали Xior по нескольким причинам:

- Xior предоставляет интерфейс, схожий с Axios, что удобно;
- мы не доверяем axios из-за его недавнего взлома¹ (31 марта 2026 года). Мы не хотим подвергать себя опасности, зная, что есть альтернативы;
- мы хотим использовать fetch API, но не хотим изобретать велосипед;
- мы хотим поддержку плагинов, при необходимости какой-то функциональности, которой нет в Xior по умолчанию. Мы не хотим реализовывать эту функциональность самостоятельно.

2.2 Разработка проекта решения

2.2.1 Упрощенная система ролей

Функции формирования нормативной базы (роль эксперта), оперативного планирования циклов (роль агронома), создания физического предприятия и расстановки теплиц (роль директора) будут консолидированы в едином пользовательском интерфейсе агронома. Роли производственного персонала не будут реализованы вовсе.

Данные решения обусловлены задачами курсового проектирования: демонстрацией алгоритмов оптимизации культурооборота без усложнения системы модулями ролевого согласования. Схема базы данных учитывает наличие ролей, не

¹ <https://www.stepsecurity.io/blog/axios-compromised-on-npm-malicious-versions-drop-remote-access-trojan>

реализуемых в рамках MVP, потому систему возможно масштабировать для точного отражения реального положения вещей.

2.2.2 Диаграммы

2.2.2.1 Диаграмма сценариев использования

На диаграмме сценариев использования (смотрите рис. 2.2.2.1.1) отображены три актора системы: Гость, Агроном и Администратор.

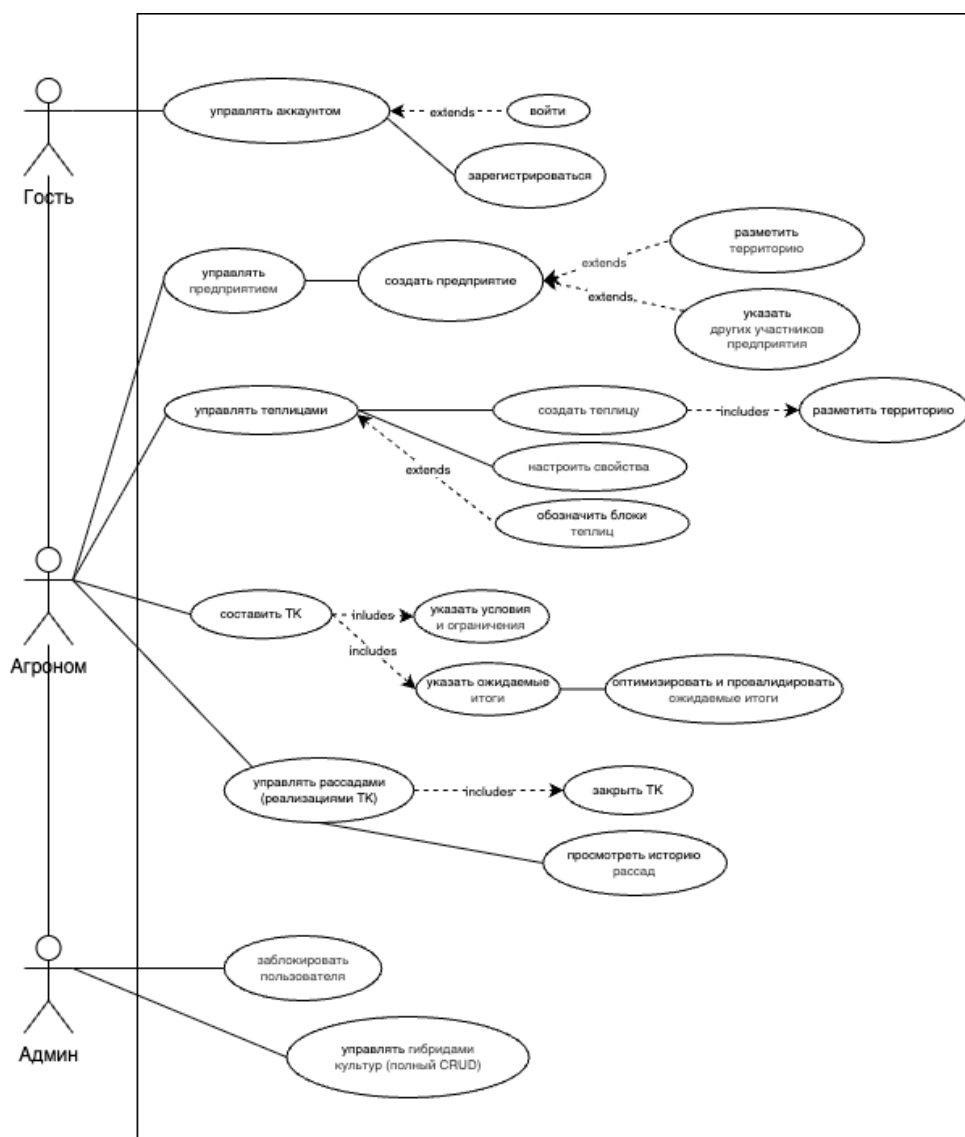
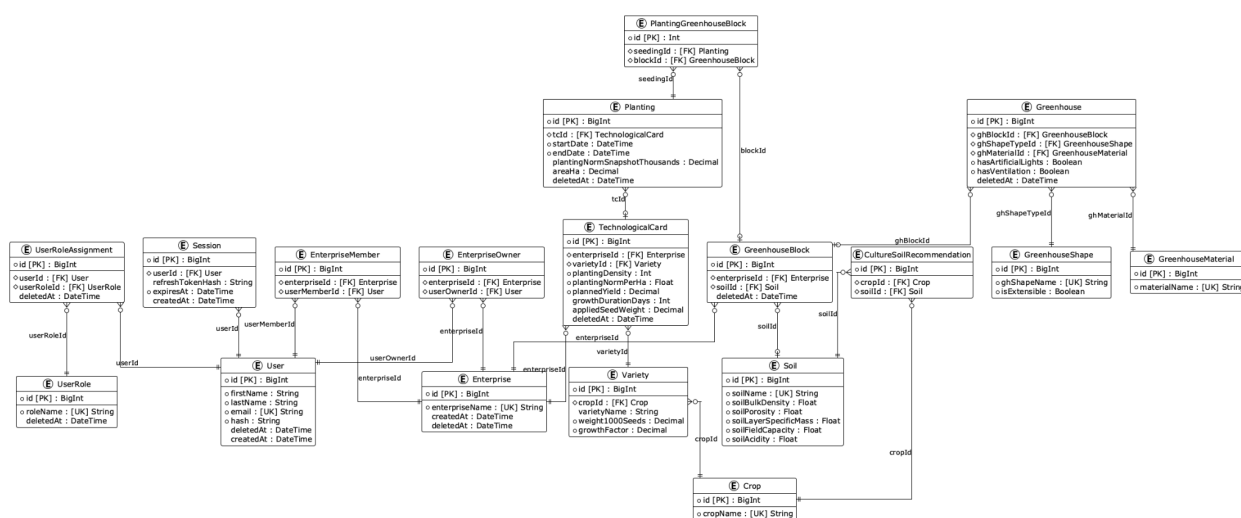


Рисунок 2.2.2.1.1 — диаграмма сценариев использования

Гость имеет возможность управлять своим аккаунтом, что включает в себя функции входа в систему, если аккаунт существует, и регистрации аккаунта.

Администратор системы выполняет функции модерации и управления справочными данными. Он может блокировать пользователей при необходимости, а также управлять гибридами культур, выполняя полный цикл операций CRUD: создание, чтение, обновление и удаление записей о гибридах.



2.2.2.3 Диаграмма последовательности

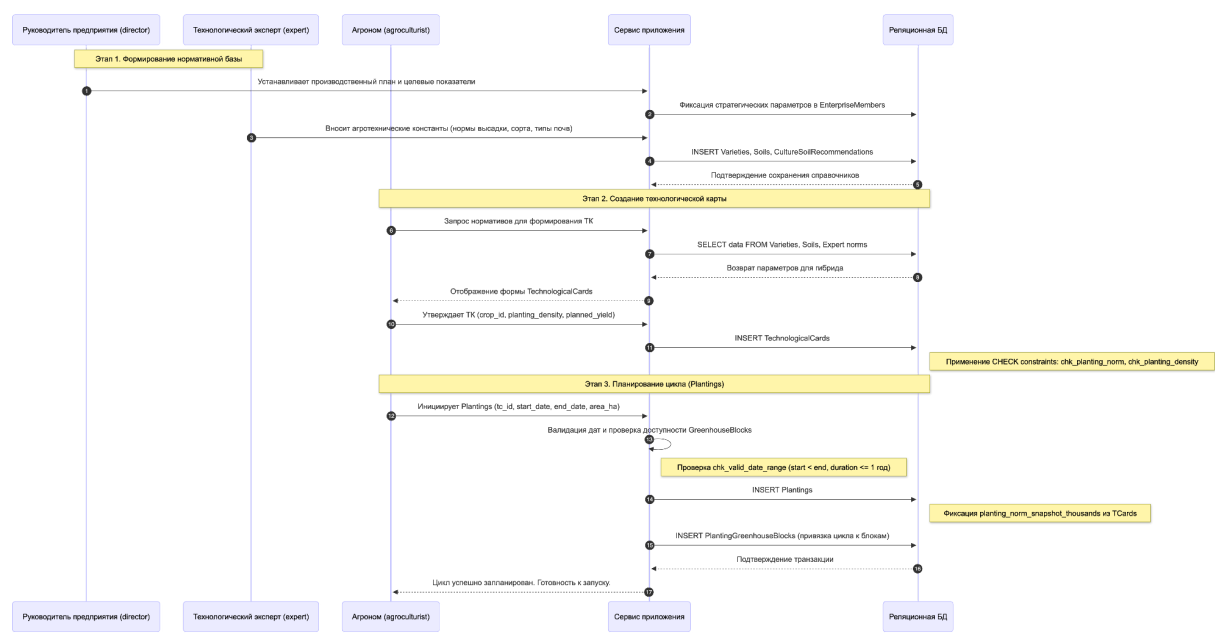


Рисунок 2.2.2.3.1 — диаграмма последовательности

3 ПРАКТИЧЕСКАЯ ЧАСТЬ

3.1 Разработка составных элементов

3.2 Установка и настройка

3.3 Тестирование

3.4 Ввод в эксплуатацию

3.5 Разработка сопроводительной документации

Заключение

Список использованных источников

- <https://radiotochki.net/blog/seedlings/rassadnyy-metod-kak-obmanut-prirodu-i-poluchit-ranniy-urozhay-dazhe-v-usloviyah-korotkogo-leta.html>
- <https://exactfarming.com/tekhkarta>
- <https://www.stepsecurity.io/blog/axios-compromised-on-npm-malicious-versions-drop-remote-access-trojan>

Приложения

Приложение А: техническое задание продукта

А.1 Общие сведения

Полное наименование проекта: Аграрио - система контроля культурооборота на ПОТК

Краткое наименование проекта: Аграрио

Домен: agrar.io

А.2 Назначение и цели создания системы

А.2.1 Назначение системы

Система предназначена для автоматизации планирования реализации технических карт на основе наличия необходимых площадей теплиц, требований к урожайности конкретных гибридов и срокам реализации ТК, а также учет предшественников для исключения развития патогенов.

В рамках проекта автоматизируется информационно-аналитическая деятельность в следующих бизнес-процессах:

- нанесение карты местности предприятия с валидацией, например, исключением постановки вершин предприятия непосредственно на водоемах;
- нанесение теплиц на карту местности предприятия с исключением выхода вершин теплицы за границы предприятия и исключением постановки вершин непосредственно на водоемах;

- объединение отдельных теплиц в блоки (группы) теплиц с возможностью редактирования и удаления отдельных блоков без нарушения исторических сведений о реализованных ТК;
- подбор одной или более ТК из банка ТК данного предприятия;
- указание требований к реализации подобранных ТК и сопоставление этих требований с реальными возможностями производства, немедленное оповещение о невозможности реализации указанных требований или о возможности их оптимизации;
- поиск наиболее оптимального способа задействовать незанятые площади теплиц и блоков теплиц с учетом всех ключевых переменных, включая предшественников и непосредственных требований по ТК;
- визуализация реализуемых и уже реализованных ТК в виде временной шкалы (диаграммы Ганта) с возможностью непосредственного редактирования сроков реализации ТК с проверкой при помощи вышеуказанного алгоритма оптимизации;
- хранение и чтение исторических сведений о реализованных ТК на временной шкале, использование этих сведений для учета предшественников.

А.2.2 Цели создания системы

Система создается с целью:

- автоматизации планирования и оптимизации культурооборота в сооружениях защищенного грунта с учетом агротехнических и ресурсных ограничений;
- повышения эффективности использования производственных площадей за счет математического алгоритма распределения циклов и минимизации простоев теплиц;
- обеспечения соблюдения фитосанитарных норм и правил чередования культур-предшественников для снижения рисков заболеваний;
- формирования единого информационного пространства для согласованной работы специалистов разного профиля и разграничения прав доступа;

- повышения качества, точности и своевременности производственного планирования за счет исключения ручных расчетов и человеческого фактора;
- предоставления инструментов визуализации и контроля выполнения графика производственных циклов в реальном времени.

А.3 Требования к системе

Система должна быть централизованной, т.е. все данные должны располагаться в центральном хранилище.

В качестве протокола взаимодействия между компонентами системы на транспортно-сетевом уровне необходимо использовать протокол TCP/IP.

Для организации информационного обмена между компонентами системы должны использоваться специальные протоколы прикладного уровня, такие как: HTTP или его расширение HTTPS, PostgreSQL wire protocol.

Для организации доступа пользователей к отчетности должен использоваться протокол презентационного уровня HTTP или его расширение HTTPS.

А.3.1 Требования к источникам данных системы

Источниками данных для системы должны быть:

- информационная система управления предприятием, содержащая данные о производственных циклах, посадках и привязке к блокам теплиц (СУБД PostgreSQL версии 17 или выше);
- информационно-справочная система, включающая нормативные параметры, технологические карты, справочники культур и гибридов (СУБД PostgreSQL версии 17 или выше);
- система управления организационной структурой, хранящая учетные записи пользователей, роли и права доступа (СУБД PostgreSQL версии 17 или выше);

- система учета инфраструктуры, содержащая реестр теплиц, блоков, их характеристик и параметров почв (СУБД PostgreSQL версии 17 или выше);
- система аналитики и отчетности, агрегирующая исторические данные циклов для формирования сводных показателей и диаграмм (СУБД PostgreSQL версии 17 или выше).

Взаимодействие с внешними информационными системами, включая сторонние API и БД, на текущем этапе разработки не предусмотрено. Система функционирует как автономное решение с внутренней архитектурой «клиент-сервер».

А.3.2 Требования к режимам функционирования системы

Система должна поддерживать следующие режимы функционирования:

- основной режим, в котором подсистемы выполняют все свои основные функции.

В основном режиме функционирования система должна обеспечивать;

- работу пользователей в режиме – 24 часов в день, 7 дней в неделю (24x7);
- выполнение своих функций – сбор, обработка и загрузка данных; хранение данных, предоставление отчетности.

А.3.3 Требования к эргономике и технической эстетике

Подсистема формирования и визуализации отчетности данных должна обеспечивать удобный для конечного пользователя интерфейс, отвечающий следующим требованиям.

В части внешнего оформления:

- должно быть обеспечено наличие локализованного (русскоязычного) интерфейса пользователя;

- должны соблюдаться требования доступности интерфейса для людей с особенностями зрения, включая различные формы дальтонизма, слабовидящих и слепых людей;
- интерфейс должен быть адаптирован для программ экранного доступа (скринридеров) в соответствии с рекомендациями и требованиями WCAG 2.1 (Web Consortium Accessibility Guidelines);
- должна присутствовать комфортная для глаза светлая тема интерфейса;
- размер шрифта обычного текста должен быть минимум 14px (пикселей), заголовков — минимум 12px;
- используемые шрифты должны быть удобными для продолжительного чтения.

В части диалога с пользователем:

- при возникновении ошибок в работе подсистемы или в вводимых пользователем данных на экран монитора должно выводиться сообщение с наименованием ошибки, по возможности с рекомендациями по её устранению на русском языке;